

Recursion Tree Method

Worked example: sorting [70, 20, 30, 40, 10, 50, 60]

NAME	G.Raju
REGISTER NO.	192411065
SUBJECT	Design and Analysis of Algorithms

Introduction

How divide-and-conquer recurrences are visualized and solved

Why Recursion Trees?

Algorithms like merge sort repeatedly split an array in half, sort each half, and merge the results. This “divide, recurse, combine” pattern is exactly what a recursion tree visualizes.

The Recursion Tree Method

A technique to solve recurrences by:

- Drawing each recursive call as a node, labeled with its subarray
- Branching every node into its recursive sub-calls
- Summing the work done across each level of the tree
- Adding all level totals for the overall running time

RECURRENCE RELATION

$$T(n) = 2T(n/2) + n$$

OUR EXAMPLE INPUT

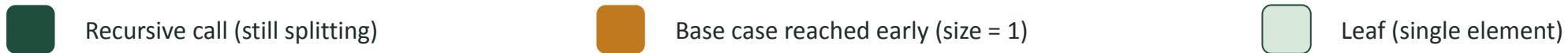
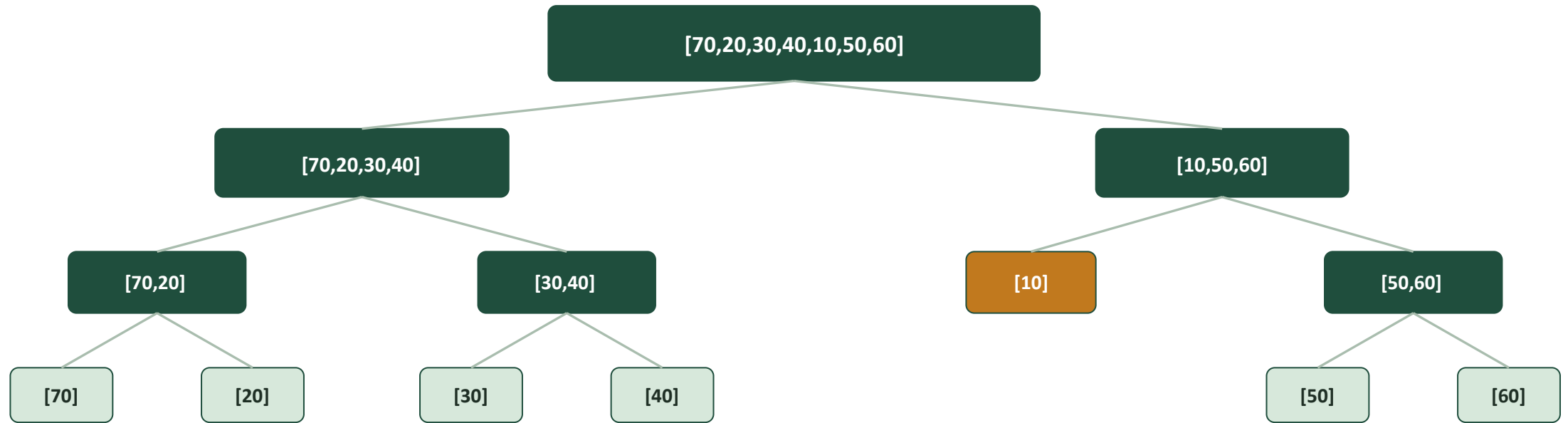
[70, 20, 30, 40, 10, 50, 60] — $n = 7$

GOAL

Build the recursion tree for our 7-element array, sum the cost level by level, and determine the overall growth rate.

Worked Example: Building the Tree

Splitting [70, 20, 30, 40, 10, 50, 60] until every subarray has one element



Notice: since $n = 7$ is not a power of 2, the split isn't perfectly even — [10] hits the base case one level early. This is normal: real inputs rarely divide evenly, but the tree still bottoms out in $\lceil \log_2 7 \rceil = 3$ levels.

Complexity Analysis

Cost per level, tree height, and the closed-form total

Level	Subarray Size	# of Nodes	Cost / Node	Total Cost
0	7	1	$\approx n$	$\approx n$
1	$\approx n/2$	2	$\approx n/2$	$\approx n$
2	$\approx n/4$	4	$\approx n/4$	$\approx n$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	1	n	1	$\approx n$

Height of Tree

$n = 7$ shrinks by $\frac{1}{2}$ each level until reaching 1 $\rightarrow$ height = $\lceil \log_2 7 \rceil = 3$

Number of Levels

levels = $\log_2 n + 1$ — for $n = 7$, that’s about 4 levels total

Total Cost

$T(n) = n \times (\log_2 n + 1) = n \log_2 n + n \rightarrow \Theta(n \log n)$

Conclusion

What the recursion tree method showed us

CLOSED FORM

$$n \log_2 n + n$$

TIME COMPLEXITY

$$\Theta(n \log n)$$

KEY TAKEAWAYS

- The recursion tree turns an abstract recurrence into a visual map of every recursive call.
- For a balanced split, each level costs $\approx n$, and there are $\approx \log_2 n$ levels.
- Our 7-element example reached its base cases within 3–4 levels, confirming the $O(n \log n)$ bound in practice.

Why This Matters

The recursion tree method is often faster and more intuitive than substitution for divide-and-conquer recurrences — it shows why an algorithm is efficient, not just what its bound is.

For algorithms like merge sort applied to our array [70, 20, 30, 40, 10, 50, 60], it confirms sorting completes in $\Theta(n \log n)$ time, far better than the $\Theta(n^2)$ of simple sorting methods.